Chapter 5: Authentication and Access Control

Flask-login handles user session management, making it easy to implement login/logout functionality and restrict access to certain pages. Combined with role-based access control (RBAC), you can create applications where different users have different permissions.

Install the required packages:

```
pip install flask flask-login flask-sqlalchemy flask-bcrypt pymysql
```

> ⚠ **Create a new MySQL database for this chapter!**
>
> You will change the user model and this would break your previous tables and entries. A clean setup prevents such issues. Keep MySQL Workbench open while working on this chapter to inspect and manipulate the data.

User model with roles

The new user model looks as follows. The class inherits from the UserMixin class of the flask-login package. This database model includes the password instead of the email (more about security later), and a role (e.g., "user" or "admin").

```
class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(250), unique=True, nullable=False)
    password = db.Column(db.String(250), nullable=False)
    role = db.Column(db.String(50), default="user", nullable=False) # "user" or
"admin"
```

How to create an admin user

Note that all users are assigned the "user" role by default. Do not let any user change themselves into an admin. Instead, go to the table in the MySQL Workbench and change that user's entry for "role" to "admin". Remember to save your changes by clicking "apply"!

Flask-login

The following code is required for Flask-login to work. The imported functions (e.g., `login_user`) are handling and hiding most of the compicated procedures so that you do not have to deal with them.

```
from flask_login import LoginManager, UserMixin, login_user, logout_user,
login_required, current_user

@login_manager.user_loader ①
def load_user(user_id):
    return User.query.get(int(user_id))

@app.route('/login', methods=["GET", "POST"])
def login():
    if request.method == "POST":
        user = User.query.filter_by(
        username=request.form.get("username")
        ).first()
        if user and user.password == request.form.get("password"): ②
            login_user(user) ③
            return redirect(url_for("home"))
    return render_template("login.html")

@app.route('/logout')
@login_required ④
def logout():
    logout_user()
    return redirect(url_for("login"))
```

① This tells Flask-login where to find a specific User object in the system.

② This checks whether the password entered in the browser is the same as the one stored in the database.

③ Calling the `login_user` function imported from the `flask_login` package and passing the `UserMixin` object handles the login logic.

4. `@login_required` is a decorator that sends an error page to the browser if the user is not logged in. Look up "Python decorators" if you want to learn more.

With this code, users only get to the homepage after successfully logging in. Note that this snippet alone will not work. You still need all the other server code:

app.py

```
from flask import Flask, render_template, request, url_for, redirect
from flask_sqlalchemy import SQLAlchemy
from flask_login import LoginManager, UserMixin, login_user, logout_user,
login_required, current_user
app = Flask(__name__)

app.config['SQLALCHEMY_DATABASE_URI'] =
'mysql+pymysql://root:mysqlrootpassword@localhost:3306/flask_app'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['SECRET_KEY'] = 'your-secret-key-here'

db = SQLAlchemy(app)

login_manager = LoginManager()
login_manager.init_app(app)
```

app.py (continued)

```
# User model with role-based access control
class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(250), unique=True, nullable=False)
    password = db.Column(db.String(250), nullable=False)
    role = db.Column(db.String(50), default="user", nullable=False) # "user" or
"admin"

with app.app_context():
    db.create_all()

# User loader - required by Flask-Login
@login_manager.user_loader
def load_user(user_id):
    return User.query.get(int(user_id))

@app.route('/register', methods=["GET", "POST"])
def register():
    if request.method == "POST":
        user = User(
        username=request.form.get("username"),
        password=request.form.get("password"),
        role="user" # Default role
        )
        db.session.add(user)
        db.session.commit()
        return redirect(url_for("login"))
    return render_template("sign_up.html")

@app.route('/login', methods=["GET", "POST"])
def login():
    if request.method == "POST":
        user = User.query.filter_by(
        username=request.form.get("username")
        ).first()
        if user and user.password == request.form.get("password"):
            login_user(user)
        return redirect(url_for("home"))
    return render_template("login.html")

@app.route('/logout')
@login_required
def logout():
    logout_user()
    return redirect(url_for("login"))

@app.route('/')
def home():
    return render_template("home.html")
```

This code stores and compares passwords in plain text!

As the developer, you can see everyone's passwords in the MySQL Workbench, and so can hackers. In one of the sections below you will learn a better, more secure approach called "hashing".

New HTML templates

You still need to create templates for `home`, `login`, and `sign_up`. The latter two are very similar to the `add_user` page that you already know, so you could use it as a starting point.

For passwords use `<input type="password">`

On the homepage, display different content depending on the user role and authentication:

- If the user is logged in with role "user", show "This is visible only to regular users."
- If the user is logged in with role "admin", show "This is visible only to admins."
- If the user is not logged in at all, show "You are not logged in."

templates/home.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Home</title>
</head>
<body>
    <nav>
        <ul>
            <li><a href="/login">Login</a></li>
            <li><a href="/register">Create account</a></li>
            {% if current_user.is_authenticated %}
            <li><a href="/logout">Logout</a></li>
            {% endif %}
        </ul>
    </nav>
    {% if current_user.is_authenticated %}
        <h1>Welcome, {{ current_user.username }}!</h1>
        {% if current_user.role == "admin" %}
            <h2>Admin Dashboard</h2>
            <p>This is visible only to admins: Manage users, settings, etc.</p>
            {% elif current_user.role == "user" %}
            <h2>User Dashboard</h2>
            <p>This is visible only to regular users.</p>
        {% endif %}
    {% else %}
    <h1>You are not logged in.</h1>
    {% endif %}
</body>
</html>
```

- `{% if current_user.is_authenticated %}` only shows the following content if the user is logged in.
- `{% if current_user.role == "admin" %}` only shows the following content if the user has the role "admin".

> **This means, the same template produces different HTML code depending on the user type.**
>
> This is different from the `@login_required` Python decorator that was introduced above. The decorator in the server prevents unauthorized users from accessing the route completely while the `current_user.is_authenticated` check conditionally adds HTML elements to the route's response.

Password hashing

Never store passwords as plain text in your database! Instead, use a one-way hash function to turn them them into a string that can still be compared to verify a password without revealing the password. Look up "hashing algorithms" if you want to learn more.

> ⚠ **Users that you have created in the previous step will no longer be able to log in and need to be deleted after this next step**

Use Bcrypt from the Flask-Bcrypt package to hash passwords. Do this everytime a password is entered (i.e., when signing up and signing in).

At the top of your `app.py` import and initialize Bcrypt:

```
from flask_bcrypt import Bcrypt
bcrypt = Bcrypt(app)
```

In the registration route, hash the new password and **only store the hashed value**:

```
@app.route('/register', methods=["GET", "POST"])
def register():
    if request.method == "POST":
        hashed_password = bcrypt.generate_password_hash(  ❶
            request.form.get("password")
        ).decode('utf-8')
        user = User(
            username=request.form.get("username"),
            password=hashed_password,  ❷
            role="user"
        )
        db.session.add(user)
        db.session.commit()
        return redirect(url_for("login"))
    return render_template("sign_up.html")
```

❶ Create the hash value for the password.

❷ Create the User object with the hash value as the password.

In the login route, verify the password:

```python
@app.route('/login', methods=["GET", "POST"])
def login():
    if request.method == "POST":
        user = User.query.filter_by(
            username=request.form.get("username")
        ).first()
        if user and bcrypt.check_password_hash(  1
            user.password, request.form.get("password")
        ):
            login_user(user)
            return redirect(url_for("home"))
    return render_template("login.html")
```

1. Compare the hashed password from the User object using the `check_password_hash` function instead of simple string comparison.

Carefully note the parameters of the `bcrypt.check_password_hash` function.

- The first parameter is the the *hashed* password retrieved from the database.
- The second parameter is the *plain* password retrieved from the HTML input field. It will be hashed inside that function.

Check inside MySQL Workbench to confirm that new passwords are no longer stored in plain text and to learn what a hashed password looks like.